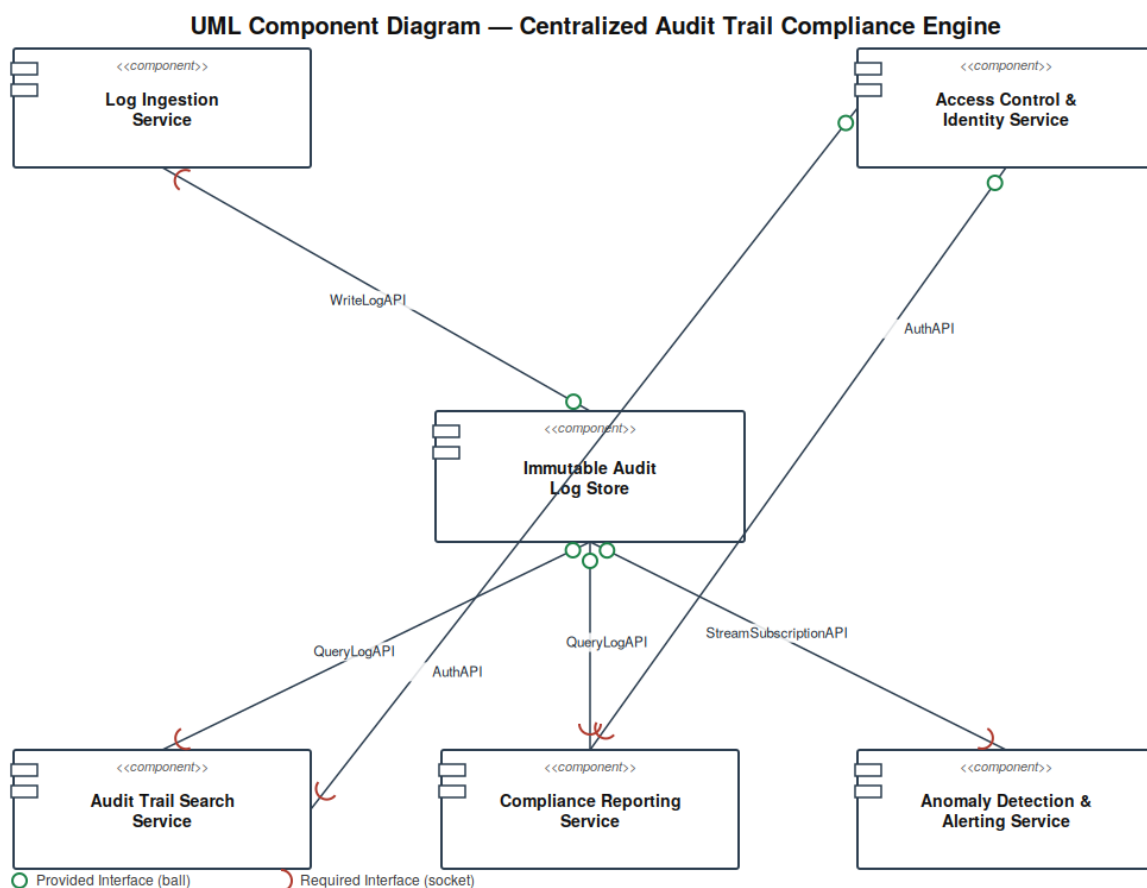


Lab 3: Component Modelling & Architectural Pattern Selection

Project: Centralized Audit Trail Compliance Engine **Domain:** Developer Tools & IT Operations **Target Actors:** Compliance Officer, Security Auditor

1. Component Diagram



Components (6, exceeds minimum of 5):

- Log Ingestion Service
- Access Control & Identity Service
- Immutable Audit Log Store
- Audit Trail Search Service

- Compliance Reporting Service
- Anomaly Detection & Alerting Service

Interfaces (4, exceeds minimum of 4 shown across 6 connections):

- WriteLogAPI — Log Ingestion Service → Immutable Audit Log Store
- QueryLogAPI — Audit Trail Search Service & Compliance Reporting Service → Immutable Audit Log Store
- StreamSubscriptionAPI — Anomaly Detection & Alerting Service → Immutable Audit Log Store
- AuthAPI — Audit Trail Search Service & Compliance Reporting Service → Access Control & Identity Service

UML notation used: green circle = provided interface (ball), red semicircle = required interface (socket), solid lines show data flow/dependency between components.

2. Written Justification

Architecture Selection: "We chose Microservices Architecture for the Centralized Audit Trail Compliance Engine."

Architectural Choice

Microservices Architecture was selected, decomposing the system into independent services — Log Ingestion, Audit Trail Search, Compliance Reporting, Anomaly Detection & Alerting, Access Control & Identity, and the Immutable Audit Log Store — each independently deployable and communicating through well-defined APIs.

Two Scenario-Related Reasons

1. **Independent, uneven scaling needs:** Log ingestion volume from CI/CD, IAM, and version-control systems can spike unpredictably, while reporting or search traffic is comparatively steady. A microservices architecture lets the Log Ingestion Service scale independently under load without over-provisioning the Reporting or Search services, which a monolithic layered system would force to scale together.
2. **Fault isolation for compliance-critical functions:** If the Anomaly Detection & Alerting service fails or is redeployed, ingestion and audit log storage must keep

running uninterrupted so no compliance data is lost. Microservices isolate failures to a single service, whereas a tightly coupled layered or client-server design risks a single failure taking down the entire audit pipeline.

Security Advantage

Each service can enforce the minimum privilege it needs — for example, only the Log Ingestion Service holds write access to the Immutable Audit Log Store, while Search and Reporting hold read-only access, and all three verify permissions through a dedicated Access Control & Identity Service. This isolates the audit trail's write path, reducing the blast radius if any single service is compromised, and keeps the tamper-evident log store from being reachable except through a narrow, audited interface.

Performance Benefit

Because ingestion, search, reporting, and alerting run as separate services, the system can allocate more compute specifically to the Log Ingestion Service during high-volume periods (e.g., CI/CD pipeline bursts) without slowing down compliance officers running reports or auditors searching logs at the same time — a level of targeted, independent performance tuning that a single-tiered client-server or layered architecture cannot offer.